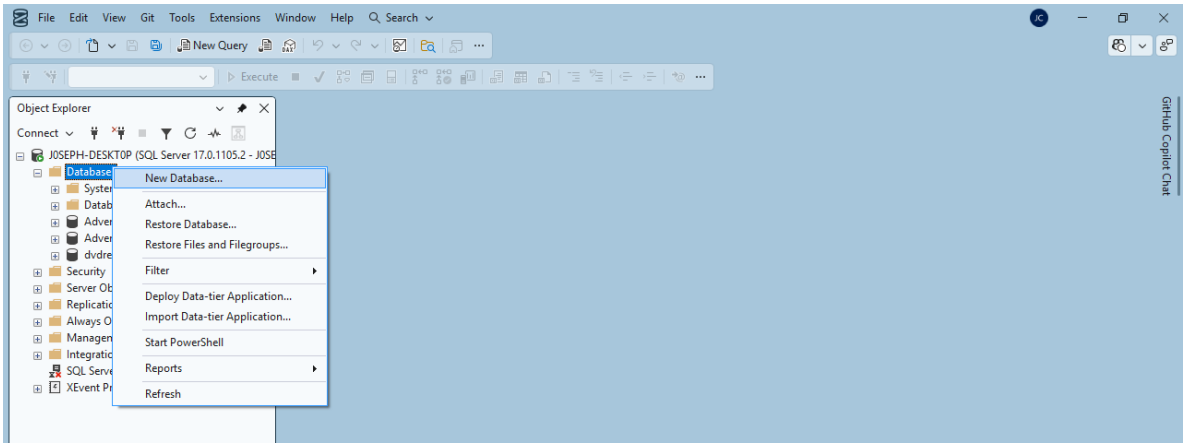
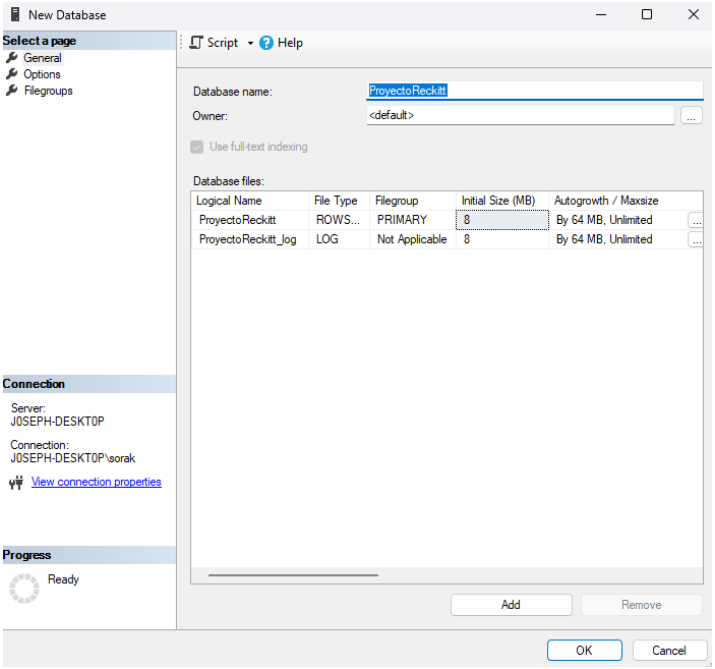
Proyecto con empresa aliada Parte 4 – Implementación de SQL

Creando la base de datos

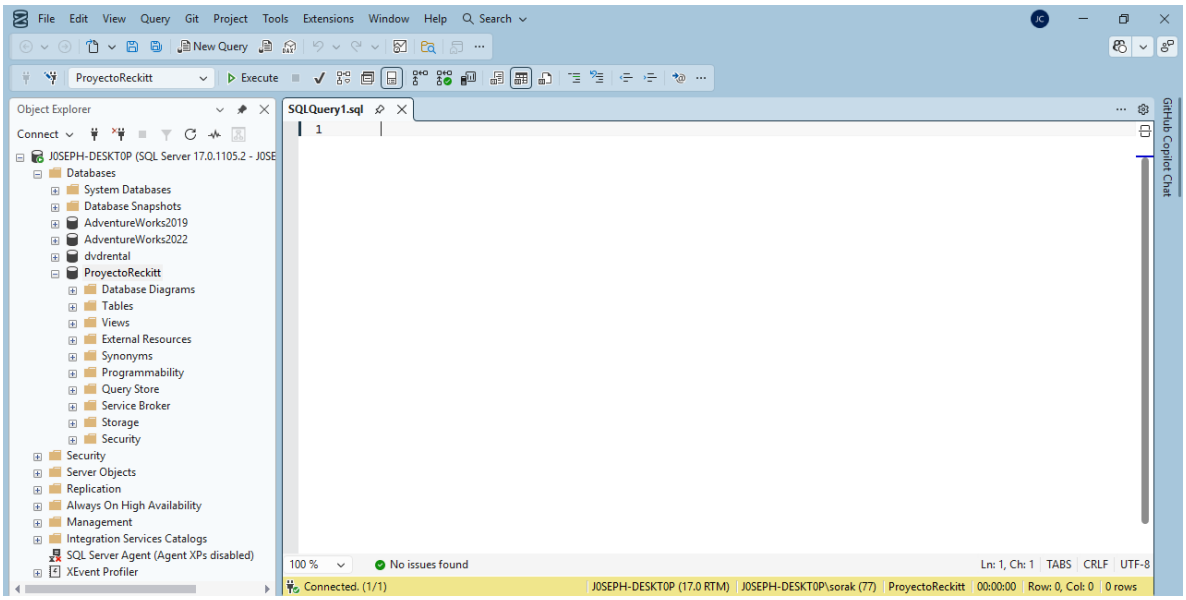
Primeramente, abrimos SSMS y creamos una nueva base de datos para almacenar los datos proporcionados, para lo cual, daremos clic derecho en “Databases” y clic en la opción “New Database...”



Una vez en el menú, se colocará el nombre de “ProyectoReckitt” a la nueva base de datos



Una vez creada la base de datos, creamos un nuevo query que esté conectado a la base de datos.



Creación de las tablas

Una vez ya creada la base de datos, el siguiente paso es crear las tablas correspondientes a cada archivo CSV y XLSX. Para ello, se definirán 5 tablas diferentes:

- *category*: con los datos de “DIM_CATEGORY.csv”
- *calendar*: con los datos de “DIM_CALENDAR.xlsx”
- *segments*: con los datos de “DIM_SEGMENT.xlsx”
- *products*: con los datos de “DIM_PRODUCT.xlsx”
- *sales*: con los datos de “FACT_SALES.csv”

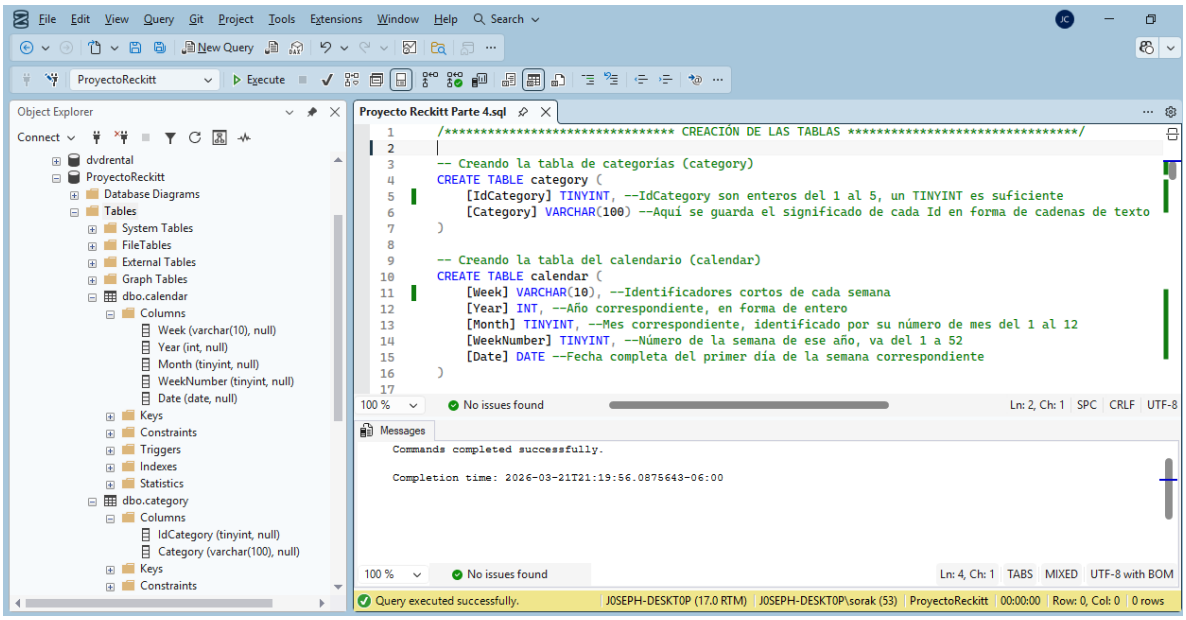
A continuación, se presentarán las columnas de cada tabla:

- *category* tiene las columnas:
    - **IdCategory**: Columna con enteros del 1 al 5 como identificadores
    - **Category**: Aquí se guarda el significado de cada Id en forma de cadenas de texto

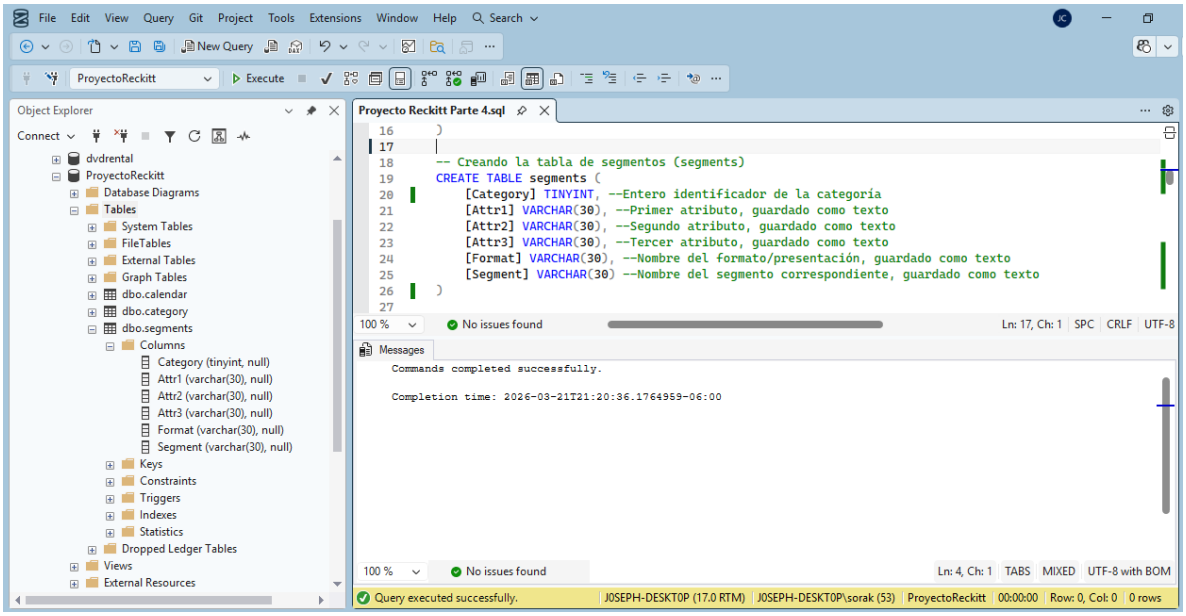
- *calendar* tiene las columnas:
    - **Week**: Identificadores cortos de cada semana
    - **Year**: Año correspondiente, en forma de entero
    - **Month**: Mes correspondiente, identificado por su número de mes del 1 al 12
    - **WeekNumber**: Número de la semana de ese año, va del 1 a 52
    - **Date**: Fecha completa del primer día de la semana correspondiente
- *segments* tiene las columnas:
    - **Category**: Entero identificador de la categoría
    - **Attr1**: Primer atributo, guardado como texto
    - **Attr2**: Segundo atributo, guardado como texto
    - **Attr3**: Tercer atributo, guardado como texto
    - **Format**: Nombre del formato/presentación, guardado como texto
    - **Segment**: Nombre del segmento correspondiente, guardado como texto
- *products* tiene las columnas:
    - **Manufacturer**: Nombre del manufacturero, guardado como texto
    - **Brand**: Nombre de la marca, guardado como texto
    - **Item**: Identificador alfanumérico del producto
    - **ItemDescription**: Descripción del producto, guardado como un texto largo
    - **Category**: Entero identificador de la categoría
    - **Format**: Nombre del formato/presentación, guardado como texto
    - **Attr1**: Primer atributo, guardado como texto
    - **Attr2**: Segundo atributo, guardado como texto
    - **Attr3**: Tercer atributo, guardado como texto
- *sales* tiene las columnas:
    - **Week**: Identificadores cortos de cada semana de la venta
    - **ItemCode**: Identificador alfanumérico del producto vendido
    - **TotalUnitSales**: Cantidad decimal que mide las ventas unitarias de esa semana
    - **TotalValueSales**: Cantidad decimal que mide las ventas monetarias de esa semana
    - **TotalUnitAvgWeeklySales**: Cantidad que mide el rendimiento histórico promedio del producto
    - **Region**: Region en la que se hizo la venta, guardada como texto

En base a esto, se crearon las tablas ya mencionadas en SQL. A continuación, se mostrará paso a paso este proceso en esta y en las siguientes capturas, donde además se verán cómo van apareciendo poco a poco las tablas en la base de datos en el menú que aparece de lado izquierdo, esto conforme se van ejecutando cada parte de la consulta.

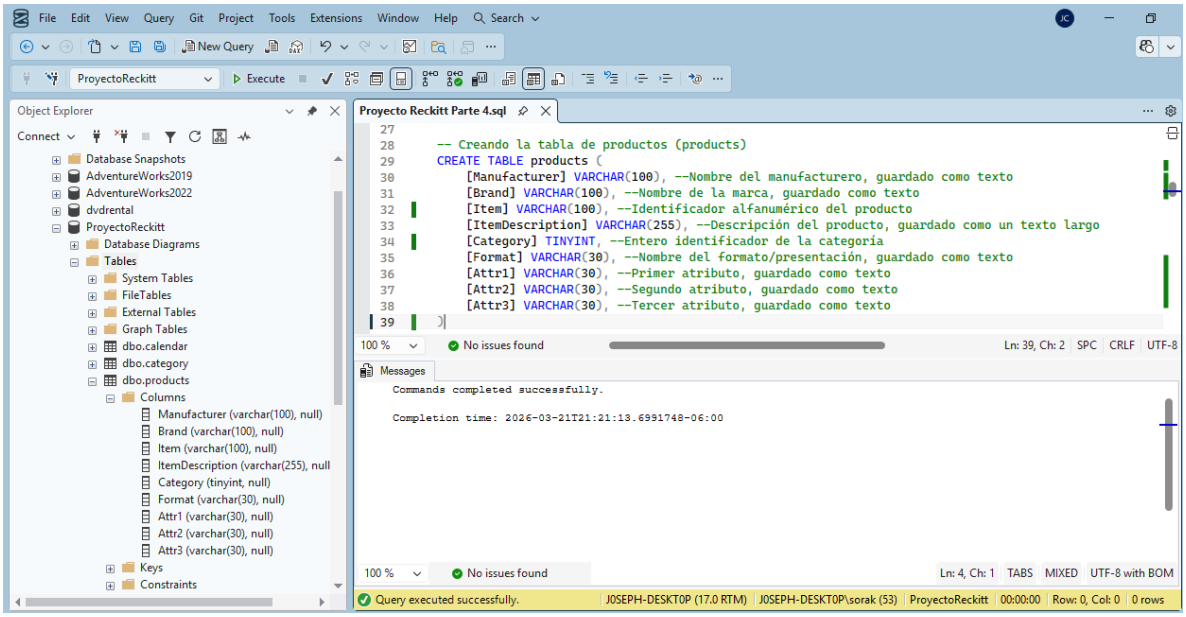
Creando las tablas de categoría y calendario:



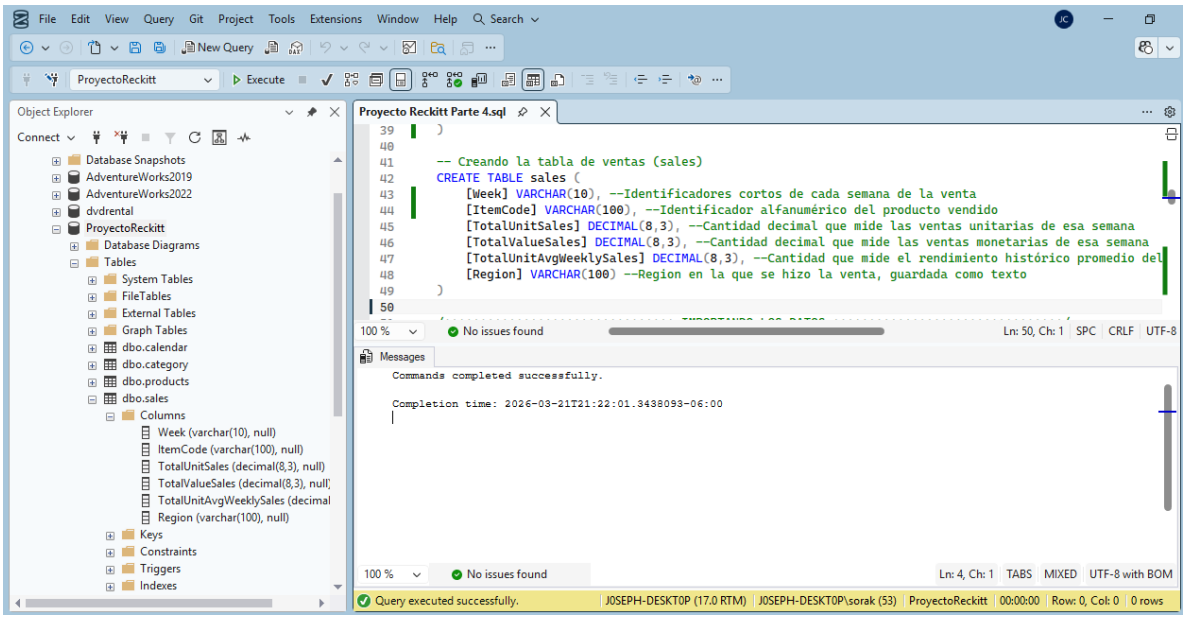
Creando la tabla de segmentos:



Creando la tabla de productos:



Creando la tabla de ventas:



Aquí, cabe destacar que se colocó para las columnas con los datos numéricos de ventas el tipo de valor DECIMAL(8,3), lo que indica que puede albergar datos con hasta 8 cifras en total y 3 cifras decimales, lo cual es suficiente para nuestros datos, ya que el máximo encontrado en estas columnas llega hasta 12236.759 (para TotalValueSales), el cual tiene 8 cifras en total y la precisión de los decimales es hasta 3 cifras.

Importando los datos

En este caso, recordemos que tenemos dos tipos de archivos proporcionados: CSV y XLSX. En el caso de los archivos CSV, es posible insertar los datos a través de un BULK INSERT. A continuación, se muestra cómo se hizo este proceso para la importación de los datos en la tabla *category*:

Proyecto Rec... Parte 4.sql*

```sql
/******************************** IMPORTANDO LOS DATOS ********************************/

-- Importar el archivo CSV
BULK INSERT category
--Ruta del archivo: esta debe de cambairse si se requiere reproducibilidad
FROM 'C:\Users\sorak\Proyecto Reckitt\DIM_CATEGORY.csv'
WITH (
    FORMAT='CSV', -- Especificamos que el formato es CSV
    FIRSTROW = 2  -- Saltar el encabezado, empezará en la segunda fila
);

SELECT *
FROM category
```

100 % | No issues found | Ln: 60, Ch: 75 | SPC | CRLF | UTF-8

Results | Messages

	IdCategory	Category
1	1	FABRIC TREATMENT and SANIT
2	2	AIR CARE
3	3	LAVAVAJILLAS
4	4	MEGA SUPERFICIES
5	5	LAVATORY CARE & BRC

Query executed successfully. | J0SEPH-DESKT0P (17.0 RTM) | J0SEPH-DESKT0P\sorak (58) | ProyectoReckitt | 00:00:00 | Row: 1, Col: 1 | 5 rows

Similarmente, este código se implementó para importar los registros en la tabla *sales*:

Proyecto Reckitt Parte 4.sql

```sql
-- Importar el archivo CSV, pero ahora para las ventas
BULK INSERT sales
--Ruta del archivo: esta debe de cambairse si se requiere reproducibilidad
FROM 'C:\Users\sorak\Proyecto Reckitt\FACT_SALES.csv'
WITH (
    FORMAT='CSV', -- Especificamos que el formato es CSV
    FIRSTROW = 2  -- Saltar el encabezado, empezará en la segunda fila
);

SELECT *
FROM sales
```

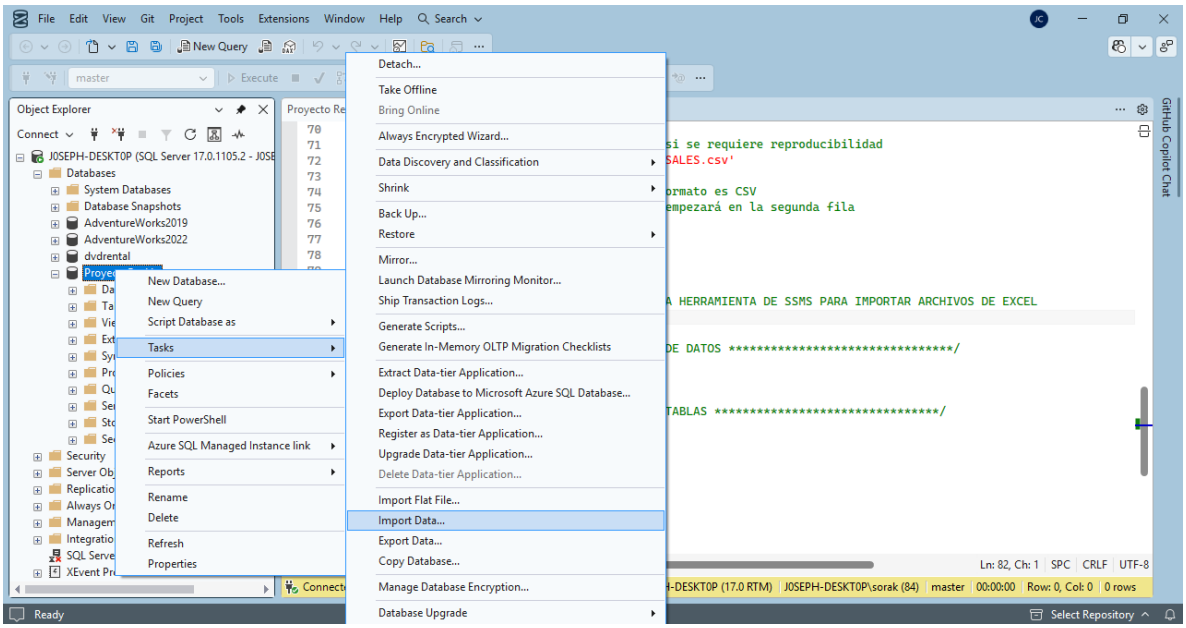
100 % | No issues found | Ln: 76, Ch: 3 | SPC | CRLF | UTF-8

Results | Messages

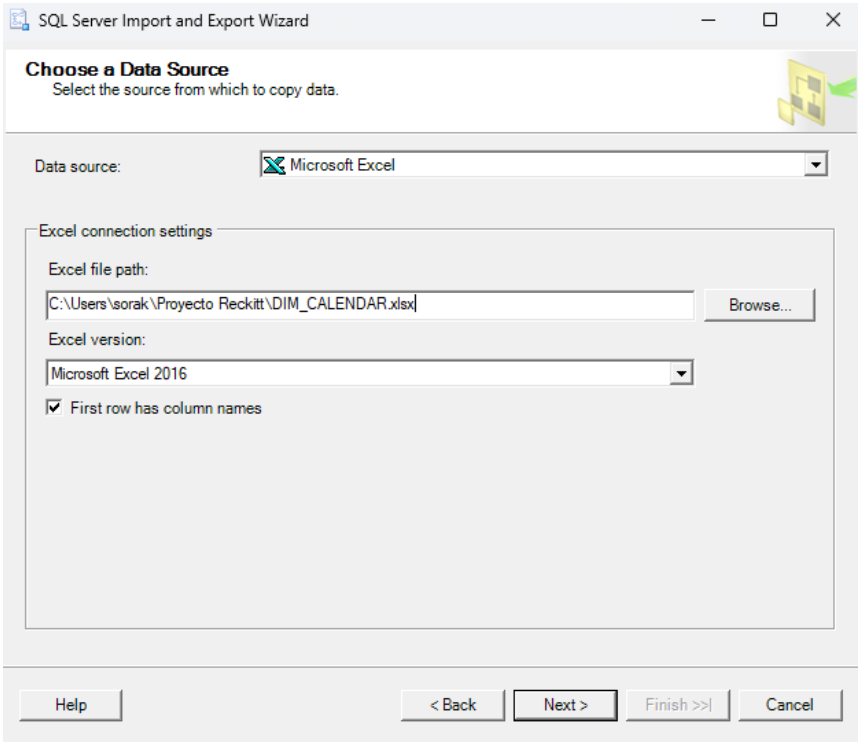
	Week	ItemCode	TotalUnitSales	TotalValueSales	TotalUnitAvgWeeklySales	Region
1	34-22	7501058792808BP2	0.006	0.139	1.000	TOTAL AUTOS AREA 5
2	34-22	7501058715883	0.487	116.519	2.916	TOTAL AUTOS AREA 5
3	34-22	7702626213774	1.391	68.453	5.171	TOTAL AUTOS AREA 5
4	34-22	7501058716422	0.022	1.481	1.833	TOTAL AUTOS AREA 5
5	34-22	7501058784353	2.037	182.839	5.375	TOTAL AUTOS AREA 5
6	34-22	7501058716064	0.005	0.679	1.250	TOTAL AUTOS AREA 5
7	34-22	7501058794963BP1	0.004	0.553	1.000	TOTAL AUTOS AREA 5
8	34-22	7501058792778BP1	0.001	0.128	1.000	TOTAL AUTOS AREA 5
9	34-22	7501058757630	0.716	71.018	4.287	TOTAL AUTOS AREA 5
10	34-22	7501058753441	2.704	61.229	11.220	TOTAL AUTOS AREA 5
11	34-22	7702626204208BP1	2.188	23.792	17.934	TOTAL AUTOS AREA 5

Query executed successfully. | J0SEPH-DESKT0P (17.0 RTM) | J0SEPH-DESKT0P\sorak (58) | ProyectoReckitt | 00:00:01 | Row: 1, Col: 1 | 122,002 rows

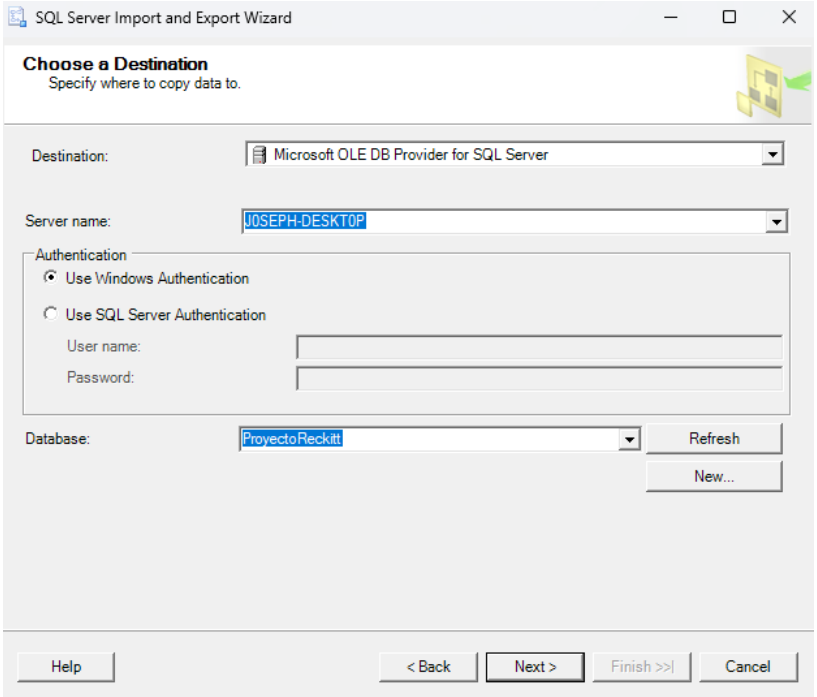
Ahora bien, para los archivos de Excel (XLSX), se aplicó otro proceso, utilizando un asistente que ofrece SMSS, al cual se accedió haciendo clic derecho en la base de datos y siguiendo los menús hasta la opción de “Import Data...”:



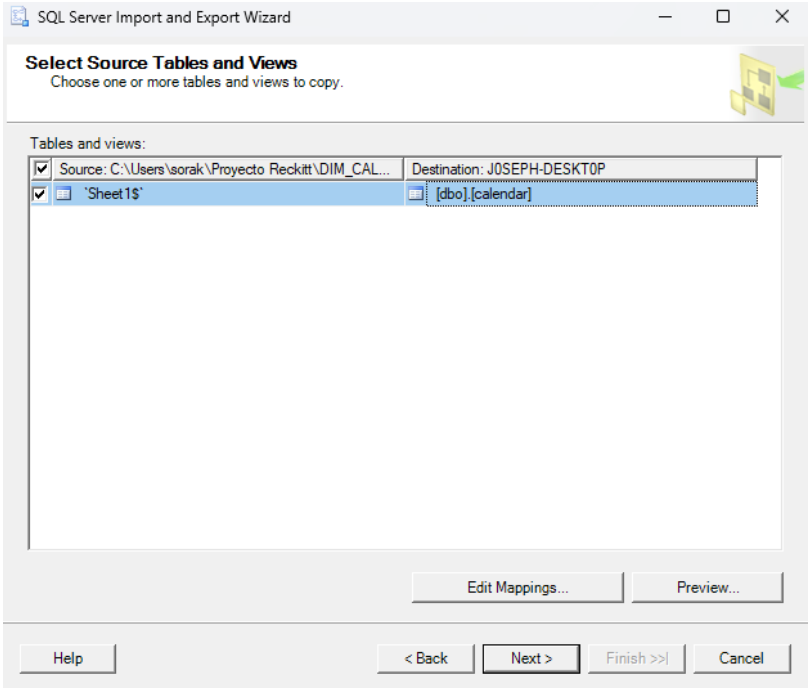
Al dar clic, se nos abre el asistente de importación y exportación de datos. En este, se especificó que la fuente de los datos era de Microsoft Excel, se especificó la ruta del archivo (para ilustrar el proceso, se mostrará lo que se realizó para la tabla *calendar*):



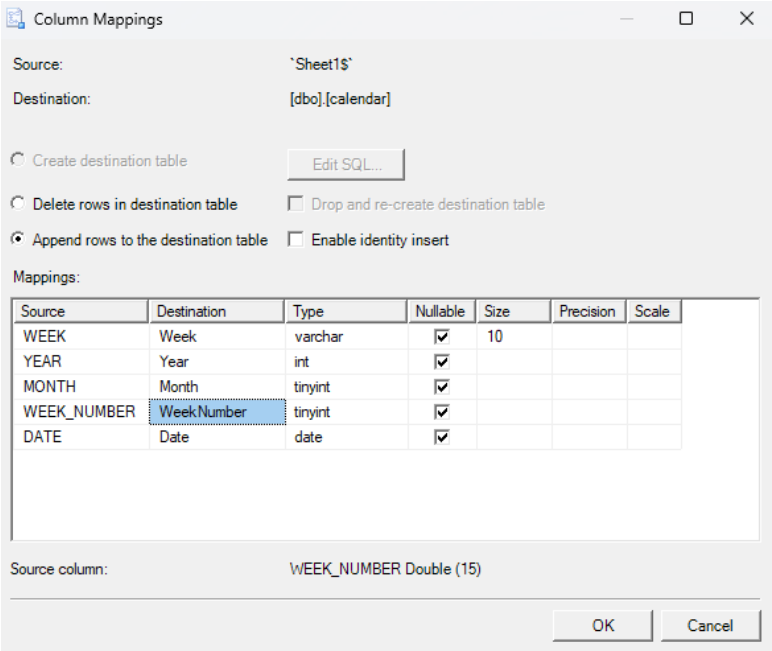
Después de darle a “Next”, se seleccionó como opción del “Destination:” la opción de “Microsoft OLE DB Provider for SQL Server”, donde ya está por defecto seleccionado mi servidor local y la base de datos de este proyecto:



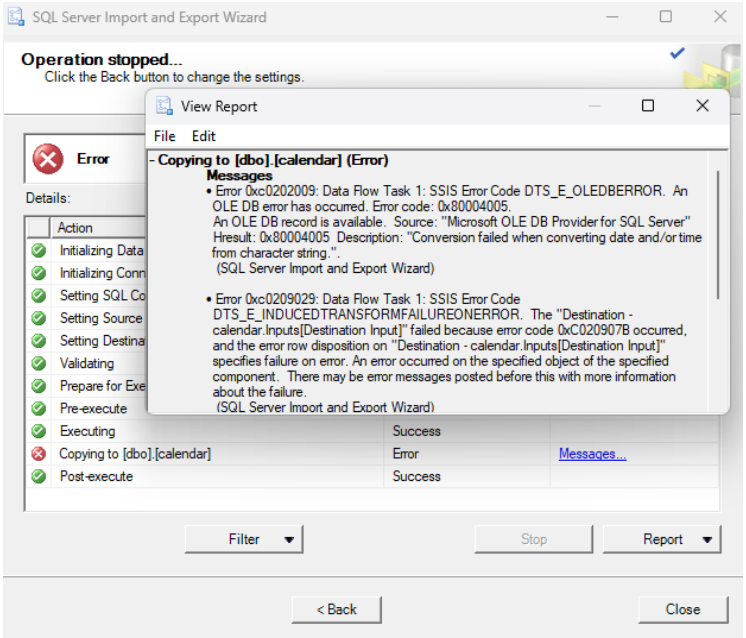
Después de dar clic en “Next” dos veces, en esta ventana se cambió el nombre de la tabla de destino a “[dbo].[calendar]”, para especificar que se insertaran los datos en esta tabla ya existente:



Antes de proseguir, es necesario revisar que el mapeo de las columnas sea correcto, para lo cual se hizo clic en “Edit Mappings”, donde originalmente la columna de “WEEK_NUMBER” estaba siendo ignorada, lo cual se corrigió inediatamente:



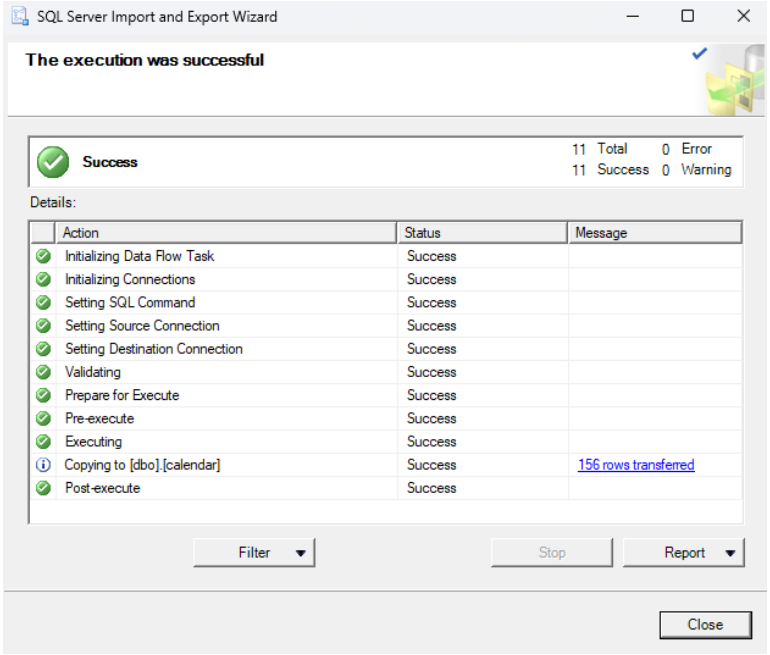
Una vez hecho esto, se prosiguió con el proceso, dando varias veces clic a “Next” y por último a “Finish”, para completar la importación. Lamentablemente hubo un error que surgió al momento de intentar convertir los datos de fechas del Excel, provocado porque el asistente tuvo un problema para convertir los datos a Fechas para el campo Date:



Por lo tanto, se decidió convertir el formato de la columna “Date” a texto, para después hacer las conversiones posteriores con consultas directamente.

```sql
-- LAS DEMÁS TABLAS FUERON IMPORTADAS CON LA HERRAMIENTA DE SSMS PARA IMPORTAR ARCHIVOS DE EXCEL

--Correción para evitar errores en el Asistente de importación de datos (requerirá convertir formato después)
ALTER TABLE calendar
ALTER COLUMN [Date] VARCHAR(10);
```

Al haber hecho esto, se realizó la importación otra vez y esta vez no hubo problemas:



Pero ahora el problema es que el campo “Date” no tenía formato de fecha, sino era solo texto, lo cual es necesario cambiar para poder realizar consultas de fechas.

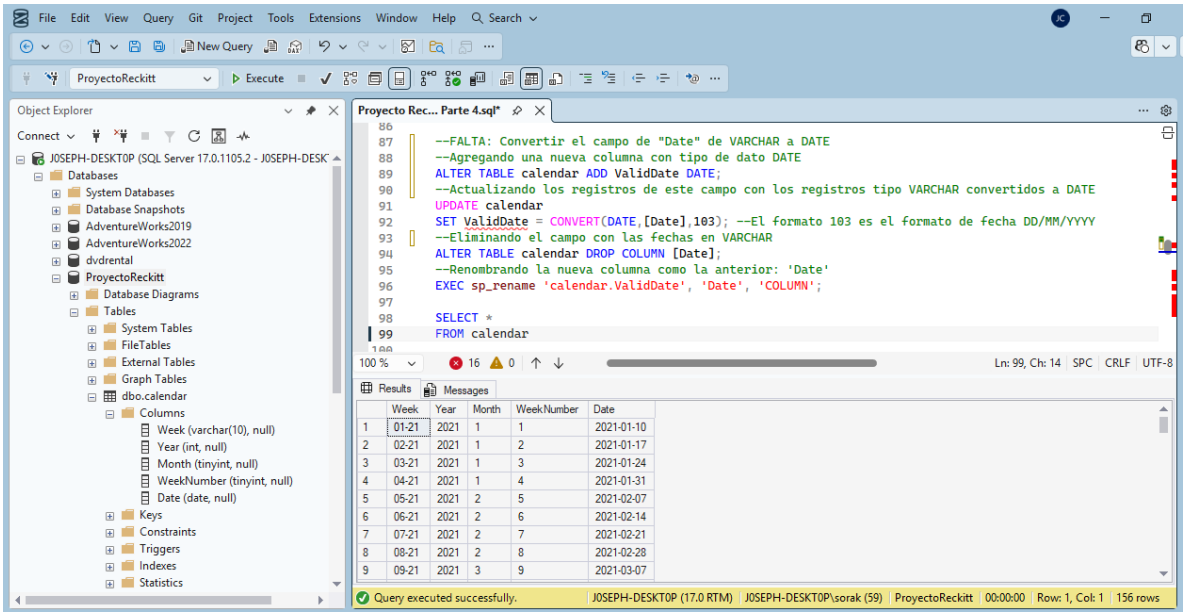
```sql
SELECT *
FROM calendar
```

Results | Messages

	Week	Year	Month	WeekNumber	Date
1	01-21	2021	1	1	10/01/2021
2	01-22	2022	1	1	09/01/2022
3	01-23	2023	1	1	09/01/2023
4	02-21	2021	1	2	17/01/2021
5	02-22	2022	1	2	16/01/2022
6	02-23	2023	1	2	16/01/2023
7	03-21	2021	1	3	24/01/2021
8	03-22	2022	1	3	23/01/2022
9	03-23	2023	1	3	23/01/2023
10	04-21	2021	1	4	31/01/2021
11	04-22	2022	1	4	30/01/2022
12	04-23	2023	1	4	30/01/2023
13	05-21	2021	2	5	07/02/2021
14	05-22	2022	2	5	06/02/2022

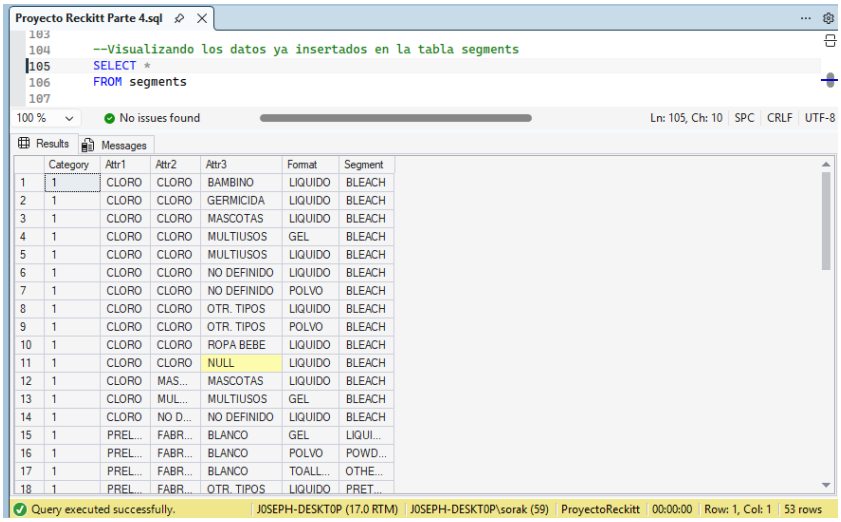
Query executed successfully. | JOSEPH-DESKTOP (17.0 RTM) | JOSEPH-DESKTOP\sorak (62) | ProyectoReckitt | 00:00:00 | Row: 1, Col: 1 | 156 rows

Para corregir esto, se siguió el siguiente procedimiento para convertirlos a DATE:



Se puede ver en el menú izquierdo, donde se despliegan las columnas de la tabla *calendar,* que el campo “Date” ahora tiene datos de tipo DATE, lo cual se ve reflejado también en la consulta al seleccionar esta tabla.

El procedimiento para importar los datos de los archivos de Excel se repitió para las tablas *segments* y *products*, donde el proceso fue más fácil y directo, ya que aquí no hubieron problemas al momento de realizar las conversiones en los tipos de datos.



Proyecto Reckitt Parte 4.sql

```sql
--Visualizando los datos ya insertados en la tabla products
SELECT *
FROM products
```

100 % No issues found Ln: 109, Ch: 10 SPC CRLF UTF-8

Results Messages

	Manufacturer	Brand	Item	ItemDescription	Category	Format
1	INDS. ALEN	CLORALEX	0000075000592	CLORALEX EL RENDIDOR BOT.PLAST. 250ML NAL. 0000075...	1	LIQUI
2	INDS. ALEN	CLORALEX	0000075000608	CLORALEX EL RENDIDOR BOT.PLAST. 500ML NAL. 0000075...	1	LIQUI
3	INDS. ALEN	CLORALEX	0000075000615	CLORALEX EL RENDIDOR BOT.PLAST. 950ML NAL. 0000075...	1	LIQUI
4	INDS. ALEN	CLORALEX	0000075000622	CLORALEX EL RENDIDOR BOT.PLAST. 2000ML NAL 000007...	1	LIQUI
5	INDS. ALEN	CLORALEX	0000075000639	CLORALEX EL RENDIDOR BOT.PLAST. 3750ML NAL 000007...	1	LIQUI
6	INDS. ALEN	CLORALEX	0000075000646	CLORALEX FANTASIA FLORAL BOT PLAST 950ML NAL 0000...	1	LIQUI
7	INDS. ALEN	CLORALEX	0000075000653	CLORALEX FANTASIA FLORAL BOT PLAST 2000 ML NAL 00...	1	LIQUI
8	INDS. ALEN	CLORALEX	0000075000677	CLORALEX FRESCURA CITRICA BOT PLAST 950ML NAL 00...	1	LIQUI
9	INDS. ALEN	CLORALEX	0000075000684	CLORALEX FRESCURA CITRICA BOT PLAST 2000ML NAL 00...	1	LIQUI
10	QUIMICA GONCAL	PATITO	0000075010539	PATITO BOTELLA 245 ML. 0000075010539	1	LIQUI
11	RECKITT	LYSOL	0019200958714	LYSOL SANITIZANTE DE ROPA CRISP LINEN BOT 1.2 LT 00...	1	LIQUI
12	P&G	TIDE	0037000018704	9TIDE LIPIADOR DE ROPA QUITAMANCHAS PLUMON 10 ML...	1	LIQUI
13	P&G	TIDE	0037000213215	9TIDE ADIVITO QUITAMANCHAS 220 GR 0037000213215	1	POLV
14	P&G	TIDE	0037000830870	9TIDE BOOST VIVID WHITE + BRIGHT BOL 18CAP = 400GR I...	1	POLV
15	P&G	TIDE	0037000889281	TIDE MULTI PURPOSE OXI STAIN REMOVER BOT 1.72KG 0...	1	POLV
16	OTHERS FABRICANTE UNIF.	OTHERS MARCA UNIF.	0041380309109	9VALUE CLORO SPECIAL BOT PLAST 3780ML 0041380309109	1	LIQUI
17	OTHERS FABRICANTE UNIF.	OTHERS MARCA UNIF.	0041380309116	SPECIAL VALUE CLORO BLANQUEADOR REGULAR BOT 3...	1	LIQUI

Query executed successfully. J0SEPH-DESKT0P (17.0 RTM) J0SEPH-DESKT0P\sorak (59) ProyectoReckitt 00:00:00 Row: 1, Col: 1 505 rows

Limpieza de datos

Nota: no se revisará la tabla *category*, ya que solo cuenta con 5 registros y ya se pudo ver en la consultas anteriores que no hay valores duplicados y nulos a tratar.

Valores nulos

En las siguientes consultas, se generará una tabla donde cada columna indica el número de datos nulos de cada tabla:

- **Tabla *segments***

Proyecto Rec... Parte 4.sql*

```sql
/********************************* LIMPIEZA DE DATOS *********************************/

--¿Hay valores nulos en las columnas de las tablas? (En category ya sabemos que no hay nulos)

--Checando valores nulos en segments
SELECT
    SUM(CASE WHEN [Category] IS NULL THEN 1 ELSE 0 END) AS CategoryNulls,
    SUM(CASE WHEN [Attr1] IS NULL THEN 1 ELSE 0 END) AS Attr1Nulls,
    SUM(CASE WHEN [Attr2] IS NULL THEN 1 ELSE 0 END) AS Attr2Nulls,
    SUM(CASE WHEN [Attr3] IS NULL THEN 1 ELSE 0 END) AS Attr3Nulls,
    SUM(CASE WHEN [Format] IS NULL THEN 1 ELSE 0 END) AS FormatNulls,
    SUM(CASE WHEN [Segment] IS NULL THEN 1 ELSE 0 END) AS SegmentNulls
FROM segments;
```

100 % 16 0 Ln: 114, Ch: 1 SPC CRLF UTF-8

Results Messages

	CategoryNulls	Attr1Nulls	Attr2Nulls	Attr3Nulls	FormatNulls	SegmentNulls
1	0	0	0	1	0	0

Aquí, solo se encontró un valor nulo en la columna de “Attr3”.

- **Tabla *calendar***

```sql
--Checando valores nulos en calendar
SELECT
    SUM(CASE WHEN [Week] IS NULL THEN 1 ELSE 0 END) AS WeekNulls,
    SUM(CASE WHEN [Year] IS NULL THEN 1 ELSE 0 END) AS YearNulls,
    SUM(CASE WHEN [Month] IS NULL THEN 1 ELSE 0 END) AS MonthNulls,
    SUM(CASE WHEN [WeekNumber] IS NULL THEN 1 ELSE 0 END) AS WeekNumberNulls,
    SUM(CASE WHEN [Date] IS NULL THEN 1 ELSE 0 END) AS DateNulls
FROM calendar;
```

	WeekNulls	YearNulls	MonthNulls	WeekNumberNulls	DateNulls
1	0	0	0	0	0

Para esta tabla, afortunadamente, no se encontraron valores nulos

- **Tabla *products***

```sql
--Checando valores nulos en products
SELECT
    SUM(CASE WHEN [Manufacturer] IS NULL THEN 1 ELSE 0 END) AS ManufacturerNulls,
    SUM(CASE WHEN [Brand] IS NULL THEN 1 ELSE 0 END) AS BrandNulls,
    SUM(CASE WHEN [Item] IS NULL THEN 1 ELSE 0 END) AS ItemNulls,
    SUM(CASE WHEN [ItemDescription] IS NULL THEN 1 ELSE 0 END) AS ItemDescriptionNulls,
    SUM(CASE WHEN [Category] IS NULL THEN 1 ELSE 0 END) AS CategoryNulls,
    SUM(CASE WHEN [Format] IS NULL THEN 1 ELSE 0 END) AS FormatNulls,
    SUM(CASE WHEN [Attr1] IS NULL THEN 1 ELSE 0 END) AS Attr1Nulls,
    SUM(CASE WHEN [Attr2] IS NULL THEN 1 ELSE 0 END) AS Attr2Nulls,
    SUM(CASE WHEN [Attr3] IS NULL THEN 1 ELSE 0 END) AS Attr3Nulls
FROM products;
```

	ManufacturerNulls	BrandNulls	ItemNulls	ItemDescriptionNulls	CategoryNulls	FormatNulls	Attr1Nulls	Attr2Nulls	Attr3Nulls
1	0	0	0	0	0	0	6	0	6

Para esta tabla, se encontraron 12 valores nulos: 6 en el campo de “Attr1” y 6 en el campo “Attr3”.

- **Tabla *sales***

```sql
--Checando valores nulos en sales
SELECT
    SUM(CASE WHEN [Week] IS NULL THEN 1 ELSE 0 END) AS WeekNulls,
    SUM(CASE WHEN [ItemCode] IS NULL THEN 1 ELSE 0 END) AS ItemCodeNulls,
    SUM(CASE WHEN [TotalUnitSales] IS NULL THEN 1 ELSE 0 END) AS ItemNulls,
    SUM(CASE WHEN [TotalValueSales] IS NULL THEN 1 ELSE 0 END) AS TotalUnitSalesNulls,
    SUM(CASE WHEN [TotalUnitAvgWeeklySales] IS NULL THEN 1 ELSE 0 END) AS TotalUnitAvgWeeklySalesNulls,
    SUM(CASE WHEN [Region] IS NULL THEN 1 ELSE 0 END) AS RegionNulls
FROM sales;
```

	WeekNulls	ItemCodeNulls	ItemNulls	TotalUnitSalesNulls	TotalUnitAvgWeeklySalesNulls	RegionNulls
1	0	0	0	0	0	0

Finalmente, en esta tabla no se encontraron valores nulos.

Ahora bien, para tratar los valores nulos encontrados, hay que considerar que solo se encontraron en los campos “Attr1” y “Attr3”. Estos campos representan atributos de los productos y se ha observado que hay un atributo llamado “NO DEFINIDO”, por lo que los valores nulos encontrados pueden ser reemplazados por este valor:

```sql
--Tratando los valores nulos
UPDATE segments --Solo hay un valor nulo en segments en la columna Attr3
SET [Attr3] = ISNULL([Attr3], 'NO DEFINIDO');
```

```
(53 rows affected)

Completion time: 2026-03-22T20:56:41.5021279-06:00
```

```sql
UPDATE products --Solo hay 12 valores nulos en la tabla products: 6 en Attr1 y 6 en Attr3
SET [Attr1] = ISNULL([Attr1], 'NO DEFINIDO'),
    [Attr3] = ISNULL([Attr3], 'NO DEFINIDO');
```

```
(505 rows affected)

Completion time: 2026-03-22T20:57:16.1412497-06:00
```

Después de actualizar las tablas, ya no hay valores nulos en la base de datos.

Registros duplicados

Primeramente, se revisarán en las tablas si hay la existencia de registros duplicados, realizando una agrupación tomando en cuenta todas las columnas, mostrando solamente aquellas que aparezcan más de una vez (por lo que serían duplicados):

- **Tabla *calendar***

```sql
--¿Hay duplicados en las tablas?
--Se agruparán los datos tomando en cuenta todas las columnas, y se mostrarán las que aparezcan más de 1 vez
SELECT *, COUNT(*) AS VecesDuplicado
FROM calendar
GROUP BY [Week], [Year], [Month], [WeekNumber], [Date]
HAVING COUNT(*) > 1;
```

Week	Year	Month	WeekNumber	Date	VecesDuplicado

Aquí, no se encontraron duplicados.

- **Tabla *segments***

```sql
SELECT *, COUNT(*) AS VecesDuplicado
FROM segments
GROUP BY [Category], [Attr1], [Attr2], [Attr3], [Format], [Segment]
HAVING COUNT(*) > 1;
```

Results | Messages

	Category	Attr1	Attr2	Attr3	Format	Segment	VecesDuplicado
1	1	CLORO	CLORO	NO DEFINIDO	LIQUIDO	BLEACH	2

En esta tabla, sí se encontró un registro que aparece repetido 2 veces.

- **Tabla *products***

```sql
SELECT *, COUNT(*) AS VecesDuplicado
FROM products
GROUP BY [Manufacturer], [Brand], [Item], [ItemDescription], [Category], [Format], [Attr1], [Attr2], [Attr3]
HAVING COUNT(*) > 1;
```

Results | Messages

Manufacturer	Brand	Item	ItemDescription	Category	Format	Attr1	Attr2	Attr3	VecesDuplicado

Aquí, no se encontraron duplicados.

- **Tabla *sales***

```sql
SELECT *, COUNT(*) AS VecesDuplicado
FROM sales
GROUP BY [Week], [ItemCode], [TotalUnitSales], [TotalValueSales], [TotalUnitAvgWeeklySales], [Region]
HAVING COUNT(*) > 1;
```

Results | Messages

Week	ItemCode	TotalUnitSales	TotalValueSales	TotalUnitAvgWeeklySales	Region	VecesDuplicado

Aquí, no se encontraron duplicados.

Para eliminar el registro duplicado encontrado, se aplicará la función ventana de ROW_NUMBER() para que realice una numeración en la partición sobre todas las columnas, por lo que si cuenta arriba de 1 en algún registro, indicará que ahí se encuentra un duplicado, por lo que se eliminarán aquellos que cumplan esta condición (para ello, es necesario utilizar una CTE).

Proyecto Rec... Parte 4.sql*

```sql
--Solo se encontró un registro duplicado en la tabla segments
WITH cte_dups AS ( --Creamos un CTE, ya que necesitamos filtrar la columna generada con ROW_NUMBER()
SELECT *, ROW_NUMBER() OVER( --Particionamos sobre todas las columnas de la tabla segments
          PARTITION BY [Category], [Attr1], [Attr2], [Attr3], [Format], [Segment]
          ORDER BY [Segment]) AS IdDuplicado
FROM segments
)
DELETE FROM cte_dups
WHERE IdDuplicado > 1; --Eliminamos los que tengan un IdDuplicado > 1, los cuales corresponden a duplicados
```

100 % 16 0 Ln: 195, Ch: 2 SPC CRLF UTF-8

Messages

```
(1 row affected)

Completion time: 2026-03-22T21:35:22.2653223-06:00
```

Definición de llaves (PRIMARY KEY y FOREIGN KEY)

Es importante definir qué columnas serán las “Primary keys” (Llaves primarias) y las “Foreign Keys” (Llaves foráneas), lo cual es clave al definir una base de datos relacional. Con el análisis previamente realizado en la parte 1 de este proyecto, donde visualizamos rápidamente los datos de cada tabla, es posible identificar qué columnas son las llaves “Primary” y “Foreign”.

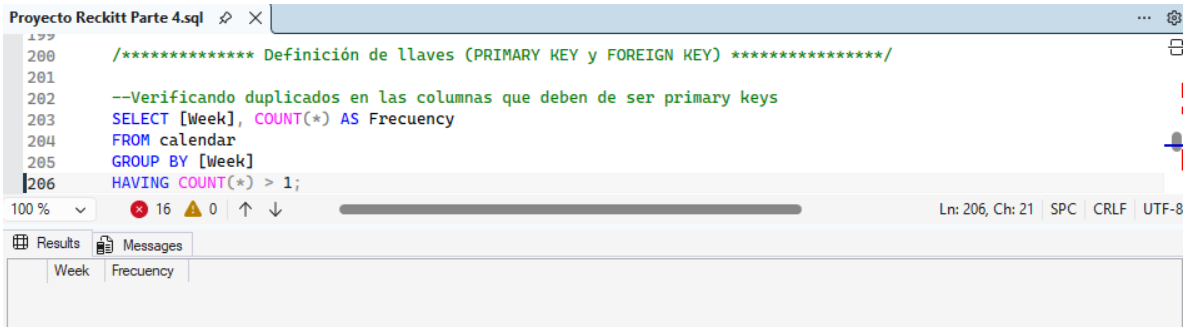
- *category* tiene las llaves:
    - ID_CATEGORY (PRIMARY KEY)
- *calendar* tiene las llaves:
    - WEEK (PRIMARY KEY)
- *segments* tiene las llaves:
    - PRIMARY KEY (CATEGORY, ATTR1, ATTR2, ATTR3, FORMAT)
- *products* tiene las llaves:
    - CATEGORY (FOREIGN KEY: REFERENCES category(ID_CATEGORY))
    - FOREIGN KEY (CATEGORY, ATTR1, ATTR2, ATTR3, FORMAT)
- *sales* tiene las llaves:
    - WEEK (FOREIGN KEY: REFERENCES calendar(WEEK))
    - ITEM_CODE (FOREIGN KEY: REFERENCES product(ITEM))

Véase que en el caso de la tabla *segments* aparece una “Primary key” con múltiples columnas. Esto pasa porque esta tabla asigna un segmento a cada combinación de las columnas de categoría, atributos y formato, formando una llave compuesta.

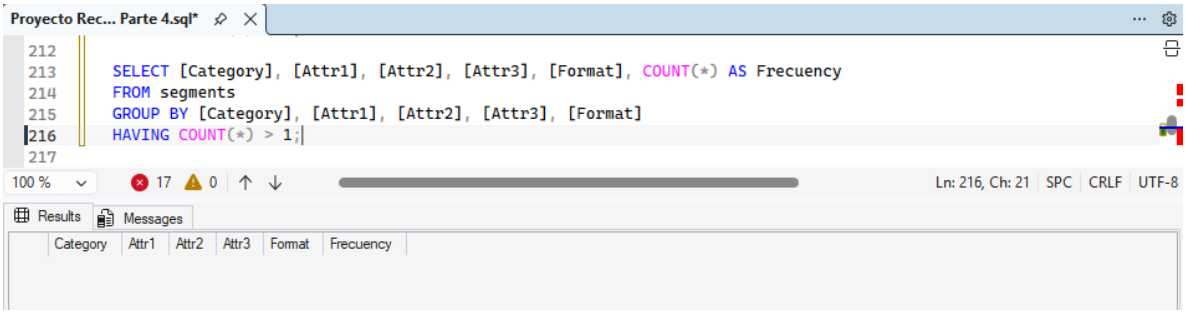
Antes de asignar las "Primary Keys", hay qe asegurarnos que no tengan valores nulos y duplicados. Anteriormente, yas e verificó que este y los demás campos no tengan valores nulos, solo falta verificar que no haya duplicados en esas columnas (los duplicados anteriormente tratados eran considerando todos los campos).

Lo anterior se puede lograr si se agrupan por las columnas que conformen estas llaves y mostrando si hay llaves que tengan más de un registro (indicando que hay llaves duplicadas):

- **Tabla *calendar***



- **Tabla *segments***



- **Tabla *products***

Al revisar todas las anteriores consultas, podemos observar que en casi todas las tablas tienen campos sin duplicados para ser asignados a llaves primarias, excepto por la tabla *products*, la cual tiene un duplicado en Item, el cual resulta ser un valor vacío (pero no es nulo, ya que está representado como “N/A”).

Hay que revisar si vale la pena conservar alguno de estos registros, ya que si no aparece el código “N/A” en la tabla donde esta llave será referenciada (*sales*):

```sql
--¿Hay algún producto en la tabla "sales" que tenga como ItemCode = 'N/A'?
SELECT [ItemCode]
FROM sales
WHERE [ItemCode] = 'N/A'
```

Results | Messages

ItemCode

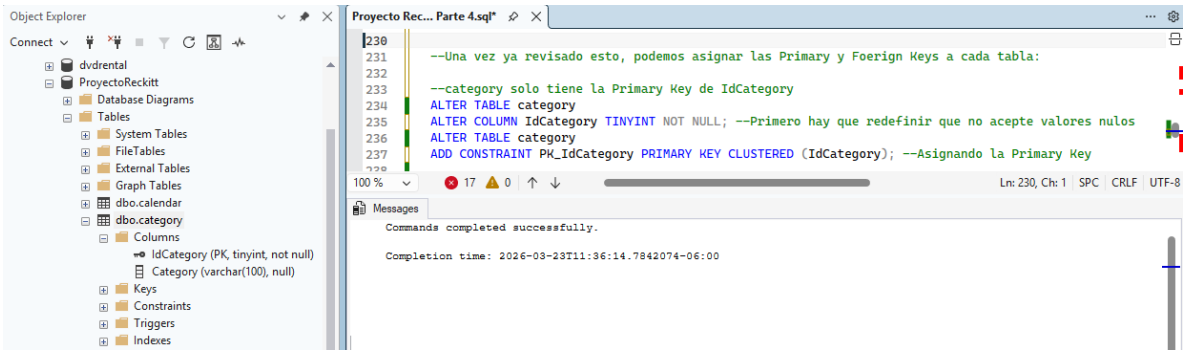
Como estos productos no están referenciados en esta tabla, podemos elinarlos directamente, ya que no nos serán de utilidad:

```sql
DELETE FROM products
WHERE [Item] = 'N/A';
```

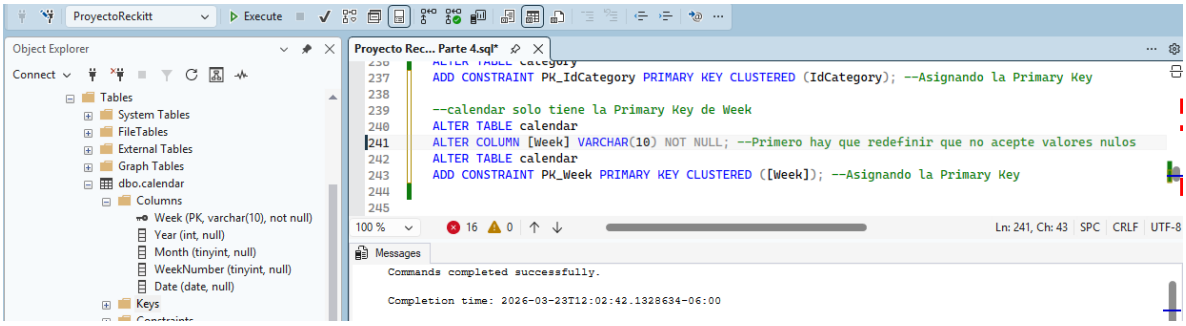
```
(2 rows affected)

Completion time: 2026-03-23T08:57:30.3106123-06:00
```

Ya que sabemos que los campos correspondientes cumplen los prerrequisitos necesarios, solo hace falta redefinirlas para que solo acepten datos no nulos y asignarles si serán “Primary Keys” o “Foreign Keys”. Esto se realizará a continuación:

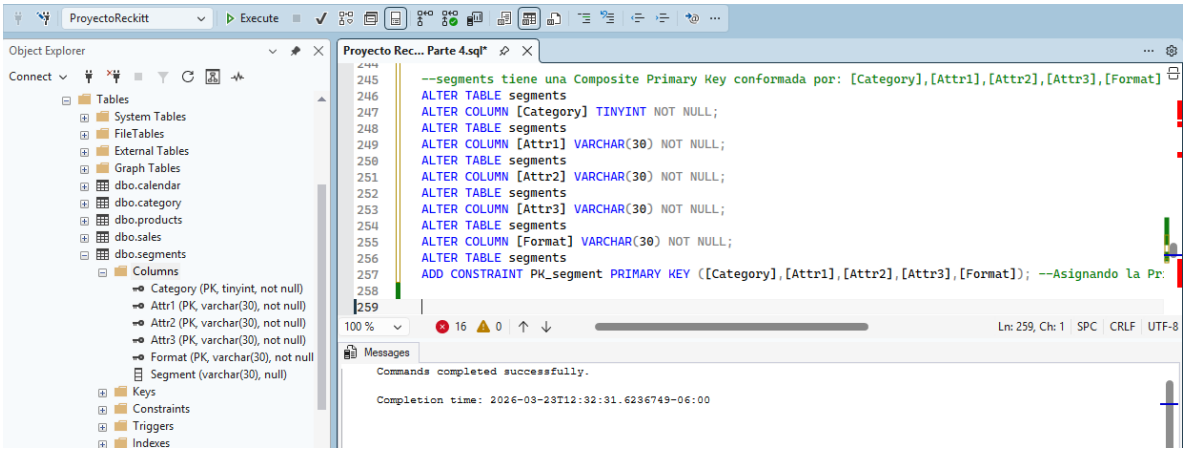
- **Tabla *category***



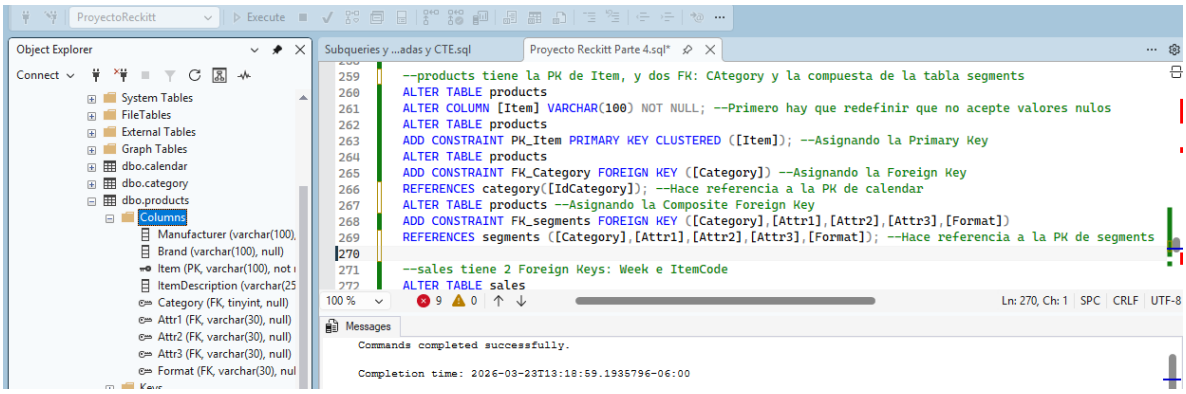
- **Tabla *calendar***



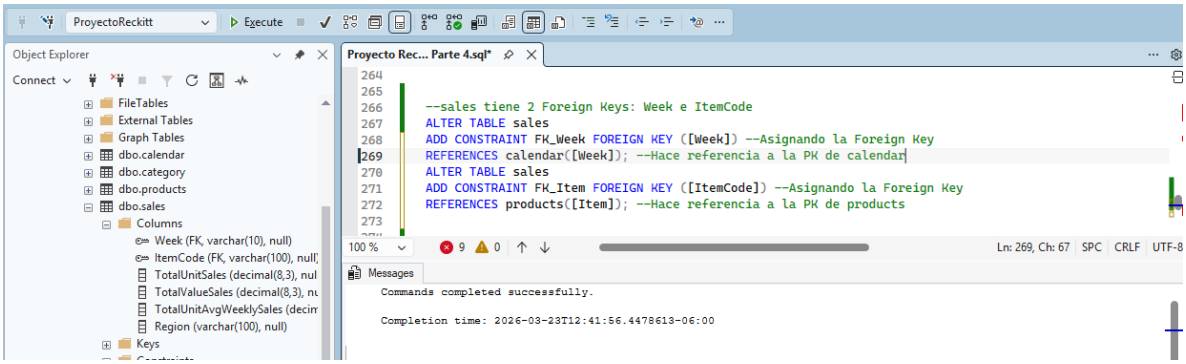
- **Tabla *segments***



- **Tabla *products***



- **Tabla *sales***


ProyectoReckitt
Execute
Object Explorer
Connect
FileTables
External Tables
Graph Tables
dbo.calendar
dbo.category
dbo.products
dbo.sales
Columns
Week (FK, varchar(10), null)
ItemCode (FK, varchar(100), null)
TotalUnitSales (decimal(8,3), nul
TotalValueSales (decimal(8,3), nu
TotalUnitAvgWeeklySales (decim
Region (varchar(100), null)
Keys
Proyecto Rec... Parte 4.sql*
--sales tiene 2 Foreign Keys: Week e ItemCode
ALTER TABLE sales
ADD CONSTRAINT FK_Week FOREIGN KEY ([Week]) --Asignando la Foreign Key
REFERENCES calendar([Week]); --Hace referencia a la PK de calendar
ALTER TABLE sales
ADD CONSTRAINT FK_Item FOREIGN KEY ([ItemCode]) --Asignando la Foreign Key
REFERENCES products([Item]); --Hace referencia a la PK de products
100 %
9
0
Ln: 269, Ch: 67
SPC
CRLF
UTF-8
Messages
Commands completed successfully.
Completion time: 2026-03-23T12:41:56.4478613-06:00

Creando Views con JOINS relevantes

Ahora que la base de datos ya está completa y preparada, podemos realizar algunos JOINS y es posible guardar los resultados de estos mismos en vistas (*views*), las cuales nos ayudarán para hacer consultas más sencillas posteriormente:

En la siguiente vista, se guardará la consulta que obtiene los datos relevantes de las tablas *products, category* y *segments*, obteniendo los detlles de todos los productos registrados:

Proyecto Rec... Parte 4.sql*

```sql
/******************************** JOINS DE TABLAS ******************************/
--Con la base de datos ya completa y preparada, podemos realizar algunos JOINS y generar views con estos
CREATE VIEW [Productos Detallados] AS
SELECT p.[Item], p.[Manufacturer], p.[Brand], p.[ItemDescription],
       c.[Category], s.[Format], s.[Segment], s.[Attr1], s.[Attr2], s.[Attr3] --Campos a visualzar
FROM products AS p
    INNER JOIN category AS c ON c.[IdCategory] = p.[Category] --Haciendo un Inner Join utilizando la identificador de categoría (PK IdCategory)
    INNER JOIN segments AS s ON p.[Category] = s.[Category] AND p.[Attr1] = s.[Attr1] AND
                                p.[Attr2] = s.[Attr2] AND p.[Attr3] = s.[Attr3] AND
                                p.[Format] = s.[Format]; --Haciendo un Inner Join utilizando la composite PK para obtener los segmentos de los produc
--Visualizando el view creado
SELECT *
FROM [Productos Detallados];
```

100 % | 9 | 0 | Ln: 292, Ch: 29 | SPC | CRLF | UTF-8

Results | Messages

	Item	Manufacturer	Brand	ItemDescription	Category	Format	Segment	Attr1	Attr2	Attr3
1	0000075000592	INDS. ALEN	CLORALEX	CLORALEX EL RENDIDOR BOT.PLAST. 250ML NAL. 000007...	FABRIC TREATMENT and SANIT	LIQUIDO	BLEACH	CLORO	CLORO	NO DEFI
2	0000075000608	INDS. ALEN	CLORALEX	CLORALEX EL RENDIDOR BOT.PLAST. 500ML NAL. 000007...	FABRIC TREATMENT and SANIT	LIQUIDO	BLEACH	CLORO	CLORO	NO DEFI
3	0000075000615	INDS. ALEN	CLORALEX	CLORALEX EL RENDIDOR BOT.PLAST. 950ML NAL. 000007...	FABRIC TREATMENT and SANIT	LIQUIDO	BLEACH	CLORO	CLORO	NO DEFI
4	0000075000622	INDS. ALEN	CLORALEX	CLORALEX EL RENDIDOR BOT.PLAST. 2000ML NAL 000007...	FABRIC TREATMENT and SANIT	LIQUIDO	BLEACH	CLORO	CLORO	NO DEFI
5	0000075000639	INDS. ALEN	CLORALEX	CLORALEX EL RENDIDOR BOT.PLAST. 3750ML NAL 000007...	FABRIC TREATMENT and SANIT	LIQUIDO	BLEACH	CLORO	CLORO	NO DEFI
6	0000075000646	INDS. ALEN	CLORALEX	CLORALEX FANTASIA FLORAL BOT PLAST 950ML NAL 0000...	FABRIC TREATMENT and SANIT	LIQUIDO	BLEACH	CLORO	CLORO	OTR. TIP
7	0000075000653	INDS. ALEN	CLORALEX	CLORALEX FANTASIA FLORAL BOT PLAST 2000 ML NAL 00...	FABRIC TREATMENT and SANIT	LIQUIDO	BLEACH	CLORO	CLORO	OTR. TIP
8	0000075000677	INDS. ALEN	CLORALEX	CLORALEX FRESCURA CITRICA BOT PLAST 950ML NAL 00...	FABRIC TREATMENT and SANIT	LIQUIDO	BLEACH	CLORO	CLORO	NO DEFI
9	0000075000684	INDS. ALEN	CLORALEX	CLORALEX FRESCURA CITRICA BOT PLAST 2000ML NAL 0...	FABRIC TREATMENT and SANIT	LIQUIDO	BLEACH	CLORO	CLORO	NO DEFI

En la siguiente vista, se guardará la consulta donde se realiza un INNER JOIN de las tablas de *sales* y *calendar*, obteniendo así la fecha en el formato adecuado (como DATE) para todas las ventas:

Proyecto Rec... Parte 4.sql*

```sql
CREATE VIEW [Ventas con Fechas] AS
SELECT ca.[Date], sa.[ItemCode], sa.[TotalUnitSales],
       sa.[TotalValueSales], sa.[TotalUnitAvgWeeklySales], sa.[Region] --Campos a visualzar
FROM sales AS sa
    INNER JOIN calendar AS ca ON ca.[Week] = sa.[Week]; --Haciendo un Inner Join utilizando la identificador de la semana (PK Week)
--Visualizando el view creado
SELECT *
FROM [Ventas con Fechas];
```

100 % | 9 | 0 | Ln: 301, Ch: 26 | SPC | CRLF | UTF-8

Results | Messages

	Date	ItemCode	TotalUnitSales	TotalValueSales	TotalUnitAvgWeeklySales	Region
1	2023-02-06	7501058751188	0.882	89.055	4.141	TOTAL AUTOS AREA 5
2	2023-02-06	7501058716095	0.134	20.655	2.436	TOTAL AUTOS AREA 5
3	2023-02-06	7501058751461	0.391	71.684	4.116	TOTAL AUTOS AREA 5
4	2023-02-06	7501058757463	0.006	0.228	3.000	TOTAL AUTOS AREA 5
5	2023-02-06	7501058757555	1.370	59.555	7.697	TOTAL AUTOS AREA 5
6	2023-02-06	7501058751195	2.161	123.963	6.670	TOTAL AUTOS AREA 5
7	2023-02-06	7501268200001	0.148	1.554	6.727	TOTAL AUTOS AREA 6
8	2023-02-06	7501268200506	0.006	0.123	3.000	TOTAL AUTOS AREA 6
9	2023-02-06	7501268200339	0.014	0.448	4.667	TOTAL AUTOS AREA 6
10	2023-02-06	7501268200049	0.626	12.132	9.788	TOTAL AUTOS AREA 6
11	2023-02-06	7501268200070	0.245	9.196	6.806	TOTAL AUTOS AREA 6
12	2023-02-06	7501268200261	0.010	0.568	5.000	TOTAL AUTOS AREA 6
13	2023-02-06	7501268200513	0.398	4.339	7.240	TOTAL AUTOS AREA 6
14	2023-02-06	7501090900585	0.074	18.726	1.480	TOTAL AUTOS AREA 6

Query executed successfully. | JOSEPH-DESKTOP (17.0 RTM) | JOSEPH-DESKTOP\sorak (91) | ProyectoReckitt | 00:00:01 | Row: 1, Col: 1 | 122,002 rows

For último, haremos un INNER JOIN de estas dos vistas, mostrando así todos los datos completos de las ventas, los productos y detalles respectivos de cada venta:

Proyecto Rec... Parte 4.sql*

```sql
--Para el siguiente Join, usaremos los dos views creados anteriormente y los juntaremos en uno solo
CREATE VIEW [Datos Completos] AS
SELECT vf.[Date], vf.[ItemCode], vf.[TotalUnitSales],
       vf.[TotalValueSales], vf.[TotalUnitAvgWeeklySales], vf.[Region],
       pd.[Manufacturer], pd.[Brand], pd.[Category], pd.[Format],
       pd.[Segment], pd.[Attr1], pd.[Attr2], pd.[Attr3] --Campos a visualzar
FROM [Productos Detallados] AS pd
    INNER JOIN [Ventas con Fechas] AS vf ON pd.[Item] = vf.[ItemCode]; --Haciendo un Inner Join utilizando la código de producto (PK Item)
--Visualizando el view creado
SELECT *
FROM [Datos Completos];
```

100 % | 9 | 0 | Ln: 311, Ch: 27 | SPC | CRLF | UTF-8

Results | Messages

	Date	ItemCode	TotalUnitSales	TotalValueSales	TotalUnitAvgWeeklySales	Region	Manufacturer	Brand	Category	Format	Segment
1	2023-02-06	7501058751188	0.882	89.055	4.141	TOTAL AUTOS AREA 5	RECKITT	VANISH	FABRIC TREATMENT and SANIT	LIQUIDO	LIQUID & G
2	2023-02-06	7501058716095	0.134	20.655	2.436	TOTAL AUTOS AREA 5	RECKITT	VANISH	FABRIC TREATMENT and SANIT	LIQUIDO	LIQUID & G
3	2023-02-06	7501058751461	0.391	71.684	4.116	TOTAL AUTOS AREA 5	RECKITT	VANISH	FABRIC TREATMENT and SANIT	LIQUIDO	LIQUID & G
4	2023-02-06	7501058757463	0.006	0.228	3.000	TOTAL AUTOS AREA 5	RECKITT	VANISH	FABRIC TREATMENT and SANIT	LIQUIDO	LIQUID & G
5	2023-02-06	7501058757555	1.370	59.555	7.697	TOTAL AUTOS AREA 5	RECKITT	VANISH	FABRIC TREATMENT and SANIT	LIQUIDO	LIQUID & G
6	2023-02-06	7501058751195	2.161	123.963	6.670	TOTAL AUTOS AREA 5	RECKITT	VANISH	FABRIC TREATMENT and SANIT	LIQUIDO	LIQUID & G
7	2023-02-06	7501268200001	0.148	1.554	6.727	TOTAL AUTOS AREA 6	BLANQUEADORA MEXICANA	LA VALENCIANA	FABRIC TREATMENT and SANIT	LIQUIDO	BLEACH
8	2023-02-06	7501268200506	0.006	0.123	3.000	TOTAL AUTOS AREA 6	BLANQUEADORA MEXICANA	LA VALENCIANA	FABRIC TREATMENT and SANIT	LIQUIDO	BLEACH
9	2023-02-06	7501268200339	0.014	0.448	4.667	TOTAL AUTOS AREA 6	BLANQUEADORA MEXICANA	LA VALENCIANA	FABRIC TREATMENT and SANIT	LIQUIDO	BLEACH
10	2023-02-06	7501268200049	0.626	12.132	9.788	TOTAL AUTOS AREA 6	BLANQUEADORA MEXICANA	LOS PATITOS	FABRIC TREATMENT and SANIT	LIQUIDO	BLEACH

Query executed successfully. | JOSEPH-DESKTOP (17.0 RTM) | JOSEPH-DESKTOP\sorak (91) | ProyectoReckitt | 00:00:02 | Row: 1, Col: 1 | 122,002 rows

Consultas relevantes e Insights claves

A continuación, se presentarán múltiples consultas a la base de datos recién creada, con el objetivo de obtener insights clave de las ventas. Para ello, se calcularán estadísticas de venta, definidas como:

- **Estadísticas de venta**: El número de ventas (frecuencia de registros), el promedio de las las ventas unitarias (TotalUnitSales) y del rendimiento histórico de las ventas (TotalUnitAvgWeeklySales), así como el promedio y la suma total de las ventas monetarias totales (TotalValueSales)

Calculando las estadísticas de venta por segmento (Segment):

```sql
SELECT [Segment],
       COUNT(*) AS Frecuencia,
       AVG(TotalUnitSales) AS AvgUnitSales,
       AVG(TotalUnitAvgWeeklySales) AS AvgUnitWeeklySales,
       AVG(TotalValueSales) AS AvgValueSales,
       SUM(TotalValueSales) AS SumValueSales
FROM [Datos Completos]
GROUP BY [Segment]
ORDER BY AVG(TotalValueSales) DESC;
```

100 % 12 0

Results Messages

	Segment	Frecuencia	AvgUnitSales	AvgUnitWeeklySales	AvgValueSales	SumValueSales
1	BLEACH	53246	5.834943	17.223652	142.382221	7581283.774
2	BAR	3639	4.992750	14.413848	66.924767	243539.230
3	LIQUID & GEL	22697	1.335608	4.955546	64.063800	1454056.069
4	POWDER	17694	1.003273	3.668503	58.495630	1035021.683
5	PRETREAT	19143	0.727143	3.857617	33.014784	632002.027
6	OTHERS	560	0.364416	2.788073	26.859669	15041.415
7	SANITIZER	5023	0.142179	1.965297	16.308121	81915.695

El segmento de “Bleach”, es aquel que cuenta con el mayor número de ventas registrado en la base de datos, así como el mayor promedio de ventas monetarias y unitarias, seguidos de los segmentos “Bar”, “Liquid & Gel”, etc. Curiosamente, “Bar” cuenta con un número mucho menor de ventas que “Bleach” y “Liquid & Gel”, pero aún así se posiciona como de los segmentos con mayor rendimiento. Esto nos indica que estos segmentos son los que tienen mejor rendimiento, siendo “Bar” especialmente el que tiene un gran potencial de crecimiento a futuro.

Calculando las estadísticas de venta por formato (Format):

```sql
--Calculando las estadísticas de venta por formato (Format)
SELECT [Format],
       COUNT(*) AS Frecuencia,
       AVG(TotalUnitSales) AS AvgUnitSales,
       AVG(TotalUnitAvgWeeklySales) AS AvgUnitWeeklySales,
       AVG(TotalValueSales) AS AvgValueSales,
       SUM(TotalValueSales) AS SumValueSales
FROM [Datos Completos]
GROUP BY [Format]
ORDER BY AVG(TotalValueSales) DESC;
```

100 % 3 0 Ln

Results Messages

	Format	Frecuencia	AvgUnitSales	AvgUnitWeeklySales	AvgValueSales	SumValueSales
1	LIQUIDO	70621	3.772702	12.235517	105.894050	7478343.726
2	GEL	26116	3.347320	8.969700	84.112750	2196688.592
3	BARRA	3639	4.992750	14.413848	66.924767	243539.230
4	POLVO	21066	0.927412	3.790848	52.655792	1109246.930
5	TOALLAS	560	0.364416	2.788073	26.859669	15041.415

Podemos observar que el formato “Liquido” es aquel con el mejor promedio de ventas monetarias, seguido de “Gel”, “Barra” y “Polvo”, quedando muy atrás “Toallas”. Los productos de formato “Gel” tienen unas ventas unitarias promedio muy similares a las de “Liquido”, pero lo interesante es que su promedio semanal histórico es menor, indicando que el formato “Liquido” ha tenido un mayor bajón de ventas que “Gel”, por lo que este último es más estable a largo plazo.

Calculando las estadísticas de venta por región (Region):

```sql
--Calculando las estadísticas de venta por región (Region)
SELECT [Region],
       COUNT(*) AS Frecuencia,
       AVG(TotalUnitSales) AS AvgUnitSales,
       AVG(TotalUnitAvgWeeklySales) AS AvgUnitWeeklySales,
       AVG(TotalValueSales) AS AvgValueSales,
       SUM(TotalValueSales) AS SumValueSales
FROM [Datos Completos]
GROUP BY [Region]
ORDER BY AVG(TotalUnitSales) DESC;
```

100 % 12 0

Results Messages

	Region	Frecuencia	AvgUnitSales	AvgUnitWeeklySales	AvgValueSales	SumValueSales
1	TOTAL AUTOS SCANNING MEXICO	20990	9.332065	10.876202	263.050467	5521429.320
2	TOTAL AUTOS AREA 2	18505	2.449063	9.756996	64.241888	1188796.150
3	TOTAL AUTOS AREA 6	16913	2.281394	10.851882	58.177589	983957.571
4	TOTAL AUTOS AREA 5	16135	2.247716	10.853989	71.480355	1153335.538
5	TOTAL AUTOS AREA 1	15781	1.634366	8.156537	45.260121	714249.979
6	TOTAL AUTOS AREA 3	16496	1.569838	10.992247	48.718194	803655.337
7	TOTAL AUTOS AREA 4	17182	1.397983	9.000717	39.427074	677435.998

Aquí se puede ver que la base de datos es bastante equilibrada en términos de las regiones, pues la diferencia entre el número de ventas de la región con mayor presencia: “Scanning Mexico” (omitiremos el “Total Autos” por simplicidad) con

20990 registros, y la de menor presencia: “Area 1” con 15781 registros, no es tan relativamente grande. La región de “Scanning Mexico” destaca mucho sobre las demás, pues su promedio de ventas monetarias es mucho más grande que las demás (263 vs. ~60-40), además de que la diferencia entre el promedio de sus ventas unitarias (9.33) no es tan grande respecto a su promedio histórico (10.87), por lo que no solo tiene muy buenas ventas, sino que se han mantenido muy estables.

Calculando las estadísticas de venta por marca (Brand). Dado que hay muchas marcas, solo se mostrará el Top 5 mejores:

```sql
--Calculando las estadísticas de venta por marca (Brand). Dado que hay muchas marcas, solo se mostrará el Top 5 mejores.
SELECT TOP(5) [Brand],
       COUNT(*) AS Frecuencia,
       AVG(TotalUnitSales) AS AvgUnitSales,
       AVG(TotalUnitAvgWeeklySales) AS AvgUnitWeeklySales,
       AVG(TotalValueSales) AS AvgValueSales,
       SUM(TotalValueSales) AS SumValueSales
FROM [Datos Completos]
GROUP BY [Brand]
ORDER BY AVG(TotalValueSales) DESC;
```

00 % 12 0

Results Messages

	Brand	Frecuencia	AvgUnitSales	AvgUnitWeeklySales	AvgValueSales	SumValueSales
1	BLANCATEL	2115	9.745294	24.994630	308.503083	652484.021
2	CLORALEX	22933	9.737089	21.067912	235.190272	5393618.509
3	VANISH	28377	1.949922	6.147886	76.410104	2168289.525
4	CLOROX	19754	2.423873	7.391577	69.441719	1371751.724
5	CLARASOL	2154	4.559482	20.281777	65.465262	141012.175

Observando el Top 5 de las mejores marcas según su promedio de ventas monetarias, tenemos a: Blancatel, Cloralex, Vanish (una de nuestras marcas de interés), Clorox y Clarasol. Blancatel y Cloralex son las marcas que actualmente dominan el mercado, con un promedio de ventas tanto monetarias como unitarias mucho mayor al de las demás. Vanish sigue siendo un buen competido contra otras marcas como Clorox, Clarasol y las demás marcas, pero se ha quedado atrás comparado con las dos anteriores.

Ahora bien, queremos saber en qué lugar se encuentra Lysol en el ranking de las marcas según sus ventas promedio, ya que no apareció en la consulta anterior:

```sql
--Supongamos que queremos saber en qué lugar se encuentra Lysol en el ranking de las marcas según sus ventas promedio
WITH cte AS ( --Si no se realiza un CTE, el ranking no funciona correctamente
SELECT [Brand],
       COUNT(*) AS Frecuencia,
       AVG(TotalUnitSales) AS AvgUnitSales,
       AVG(TotalUnitAvgWeeklySales) AS AvgUnitWeeklySales,
       AVG(TotalValueSales) AS AvgValueSales,
       SUM(TotalValueSales) AS SumValueSales,
       RANK() OVER(ORDER BY AVG(TotalValueSales) DESC) AS RankAvgSales,
       RANK() OVER(ORDER BY SUM(TotalValueSales) DESC) AS RankSumSales
FROM [Datos Completos]
GROUP BY [Brand]
)
SELECT *
FROM cte
WHERE [Brand] = 'LYSOL';
```

00 % 12 0

Results | Messages

	Brand	Frecuencia	AvgUnitSales	AvgUnitWeeklySales	AvgValueSales	SumValueSales	RankAvgSales	RankSumSales
1	LYSOL	3449	0.153519	1.887263	20.457334	70557.346	13	10

Después de la búsqueda, se encontró que Lysol se encuentra en el puesto 13 del ranking de las marcas según su promedio de ventas monetarias en el puesto 10 según la suma total de sus ventas. Considerando que hay 24 marcas en total en la base de datos, podemos decir que el rendimiento de esta marca no es el peor, pero es mejorable. Esto se nota también con su promedio de ventas monetarias siendo ~4 veces menor que el de Vanish.

Agrupaciones múltiples

Calculando las estadísticas de venta por Región y Segmento:

```sql
--Calculando las estadísticas de venta por Región y Segmento
SELECT TOP (10)
       [Region],
       [Segment],
       COUNT(*) AS Frecuencia,
       AVG(TotalUnitSales) AS AvgUnitSales,
       AVG(TotalUnitAvgWeeklySales) AS AvgUnitWeeklySales,
       AVG(TotalValueSales) AS AvgValueSales,
       SUM(TotalValueSales) AS SumValueSales
FROM [Datos Completos]
GROUP BY [Region], [Segment]
ORDER BY AVG(TotalValueSales) DESC;
```

100 % 12 0

Results | Messages

	Region	Segment	Frecuencia	AvgUnitSales	AvgUnitWeeklySales	AvgValueSales	SumValueSales
1	TOTAL AUTOS SCANNING MEXICO	BLEACH	9757	15.921257	18.180764	388.504825	3790641.580
2	TOTAL AUTOS SCANNING MEXICO	LIQUID & GEL	3700	4.096531	4.910720	196.494038	727027.941
3	TOTAL AUTOS SCANNING MEXICO	BAR	620	14.652106	12.970933	196.402516	121769.560
4	TOTAL AUTOS SCANNING MEXICO	POWDER	2915	3.044916	3.538494	177.533720	517510.795
5	TOTAL AUTOS AREA 2	BLEACH	8113	4.556583	16.833136	106.323650	862603.778
6	TOTAL AUTOS AREA 6	BLEACH	7432	4.377094	19.206606	102.043338	758386.089
7	TOTAL AUTOS AREA 5	BLEACH	6713	3.948283	17.625490	100.263115	673066.297
8	TOTAL AUTOS SCANNING MEXICO	PRETREAT	3158	2.203873	3.987036	100.063620	316000.912
9	TOTAL AUTOS SCANNING MEXICO	OTHERS	80	1.275425	2.897237	94.008812	7520.705
10	TOTAL AUTOS AREA 1	BLEACH	6683	3.168397	13.565432	77.679175	519129.933

Observando el top 10 de los mejores grupos de regiones y segmentos, observamos de nuevo que la región “Scanning Mexico” supera con creces a las demás, solo siendo en algunos segmentos por las ventas de productos de “Bleach” en las Areas 2, 6 y 5.

Esto, junto con la evidencia de que el grupo con mejor rendimiento es el de la región “Scanning Mexico” con el formato “Bleach”, afirma que este formato es el más fuerte del mercado en casi todas las regiones, y que sí se fuera a apostar por otro formato sería por “Liquid & Gel” y “Bar” en la región “Scanning Mexico”. Cabe destacar que este último formato ha estado teniendo un rendimiento más grande de lo normal, pues sus ventas unitarias promedio son de 14.65, superando su rendimiento histórico ha sido de 12.97, algo que casi no pasa en estos datos de ventas, por lo que apostar por este formato en esta región puede ser también muy rentable a largo plazo.

Calculando las estadísticas de venta por Segmento y Formato:

```sql
--Calculando las estadísticas de venta por Segemento y Formato
SELECT TOP (10)
       [Segment],
       [Format],
       COUNT(*) AS Frecuencia,
       AVG(TotalUnitSales) AS AvgUnitSales,
       AVG(TotalUnitAvgWeeklySales) AS AvgUnitWeeklySales,
       AVG(TotalValueSales) AS AvgValueSales,
       SUM(TotalValueSales) AS SumValueSales
FROM [Datos Completos]
GROUP BY [Segment], [Format]
ORDER BY AVG(TotalValueSales) DESC;
```

100 % 12 0

Results Messages

	Segment	Format	Frecuencia	AvgUnitSales	AvgUnitWeeklySales	AvgValueSales	SumValueSales
1	BLEACH	LIQUIDO	36515	6.578687	20.151701	165.766974	6052981.092
2	BLEACH	GEL	13359	5.141231	12.448812	108.846278	1454077.435
3	LIQUID & GEL	LIQUIDO	9940	1.164728	4.479579	71.573934	711444.912
4	BAR	BARRA	3639	4.992750	14.413848	66.924767	243539.230
5	POWDER	POLVO	17694	1.003273	3.668503	58.495630	1035021.683
6	LIQUID & GEL	GEL	12757	1.468755	5.326409	58.212052	742611.157
7	PRETREAT	LIQUIDO	19143	0.727143	3.857617	33.014784	632002.027
8	OTHERS	TOALLAS	560	0.364416	2.788073	26.859669	15041.415
9	BLEACH	POLVO	3372	0.529344	4.432831	22.012232	74225.247
10	SANITIZER	LIQUIDO	5023	0.142179	1.965297	16.308121	81915.695

Analizando estos grupos, podemos ver que casi todos los productos de “Bleach” encabezan este top por rendimiento de ventas, a excepción de aquellos con formato de “Polvo”, lo que indica que si se realiza un producto del segmento “Bleach”, debe de hacerse de preferencia en formato “Liquido” o “Gel”, y si se realizará un producto de formato “Polvo”, debería de hacerse mejor en el segmento “Powder” directamente. El formato “Barra” y segmento “Barra”, de formato “Liquido” y segmento “Liquid & Gel” son otros de los mejores grupos encontrados en esta base de datos.

Consultas más elaboradas

Mostrando el mejor producto de cada marca, mostrando sus estadisticas de ventas. Para solo desplegar información importante, solamente se seleccionarán los productos de las marcas Vanish y Lysol (las marcas de interés) y las otras marcas del TOP 5 mejores ventas: Blancatel, Cloralex, Clorox, Clarasol:

```sql
WITH cte_items AS ( --CTE con las estadísticas de ventas de cada producto
    SELECT [Brand],
           [ItemDescription],
           [Segment],
           AVG(TotalUnitSales) AS AvgUnitSales,
           AVG(TotalUnitAvgWeeklySales) AS AvgUnitWeeklySales,
           AVG(TotalValueSales) AS AvgValueSales,
           SUM(TotalValueSales) AS SumValueSales
    FROM [Datos Completos]
    GROUP BY [Brand],[ItemDescription],[Segment]
),
cte_ranked AS ( --Realizando otro CTE para poder aplicar un RANK particionado por marca, ya que después se utilziará esta nueva columna
    SELECT *,
           RANK() OVER(PARTITION BY [Brand] ORDER BY AvgValueSales DESC) AS RankBrand
    FROM cte_items
)
SELECT *
FROM cte_ranked --Se seleccioanrá solo el mejor producto por marca (RankBrand = 1) solo de las marcas correspondientes
WHERE RankBrand = 1 AND [Brand] IN ('VANISH','LYSOL','BLANCATEL','CLORALEX','CLOROX','CLARASOL')
ORDER BY AvgValueSales DESC;
```

100 % 12 0 Ln: 440, Ch: 29

Results Messages

	Brand	ItemDescription	Segment	AvgUnitSales	AvgUnitWeeklySales	AvgValueSales	SumValueSales	RankBrand
1	CLORALEX	CLORALEX EL RENDIDOR BOT.PLAST. 3750ML NAL 000007500...	BLEACH	50.597173	60.226976	2047.945771	1146849.632	1
2	BLANCATEL	BLANCATEL CONCENTRADO BOT PLAST 3750ML NAL 7501025...	BLEACH	27.880732	43.184630	876.920057	491075.232	1
3	CLOROX	CLOROX REGULAR CONCENTRADO BOTPLAST.3800ML 75010...	BLEACH	19.873625	26.395200	714.276391	399994.779	1
4	VANISH	VANISH LIQUIDO BOTELLA 925ML NAL. 7503002319529	LIQUID & GEL	5.245885	7.405821	289.469585	162102.968	1
5	CLARASOL	CLARASOL BLANQUEADOR REGULAR BOT PLAST 1LT NAL 074...	BLEACH	12.806260	42.789173	124.889880	69938.333	1
6	LYSOL	LYSOL CRISP LINEN SANITIZANTE DE ROPA BOT 2.66 LT 75010...	SANITIZER	0.243630	1.988458	61.341805	34351.411	1

Analizando el mejor producto de cada marca, el mejor producto de Vanish es el “Vanish Liquido Botella 925 ml”, con un promedio de ventas monetarias de 289.47, posicionándose como el 4to mejor producto de las 5 marcas con mejor rendimiento de ventas, siendo superados por el mejor producto de: Cloralex (“Cloralex El Rendidor 3750 ml”), Blancatel (“Blancatel Concentrado 3750 ml”) y Clorox (“Clorox Regular Concentrado 3800 ml”). Todos los anteriores productos son del segmento “Bleach”, además de que tienen presentaciones de más de 3 litros, lo que podría ser un patrón de que los consumidores prefieren presentaciones de alto volumen como estas. El mejor producto de Lysol es “Lysol Crisp Linen Sanitizante de Ropa 2.66 lt”, lo que vuelve a indicar que las presentaciones de alto volumen son muy demandadas.

Vamos a utilizar las fechas para crear una categoría con fechas según el trimestre al que correspondan

La división trimestral estándar corporativa está definida como:

- Q1: Ene-Mar
- Q2: Abr-Jun
- Q3: Jul-Sep
- Q4: Oct-Dic

```sql
--Necesitamos saber primero cuál es el rango de fechas disponibles
SELECT MIN([Date]) AS FechaMin, MAX([Date]) AS FechaMax
FROM [Datos Completos]; --Las fechas aparecerán en formato YYYY-MM-DD
```

100 % 12 0

Results | Messages

	FechaMin	FechaMax
1	2022-01-09	2023-07-17

Podemos ver que tenemos ventas del 9 de Enero del 2022 al 17 de Julio del 2023, por lo que podemos dividir las ventas del del Q1 de 2022 al Q3 de 2023. A continuación, se agruparán las ventas por trimestres, pero solo de las marcas Vanish y Lysol.

```sql
WITH cte_trimesters AS (
SELECT *,
       CASE
            WHEN [Date] BETWEEN '2022-01-01' AND '2022-03-31' THEN 'Q1-2022'
            WHEN [Date] BETWEEN '2022-04-01' AND '2022-06-30' THEN 'Q2-2022'
            WHEN [Date] BETWEEN '2022-07-01' AND '2022-09-30' THEN 'Q3-2022'
            WHEN [Date] BETWEEN '2022-10-01' AND '2022-12-31' THEN 'Q4-2022'
            WHEN [Date] BETWEEN '2023-01-01' AND '2023-03-31' THEN 'Q1-2023'
            WHEN [Date] BETWEEN '2023-04-01' AND '2023-06-30' THEN 'Q2-2023'
            ELSE 'Q3-2023' END AS Trimester
FROM [Datos Completos]
)
SELECT [Trimester],
       COUNT(*) AS Frecuencia,
       AVG(TotalUnitSales) AS AvgUnitSales,
       AVG(TotalUnitAvgWeeklySales) AS AvgUnitWeeklySales,
       AVG(TotalValueSales) AS AvgValueSales,
       SUM(TotalValueSales) AS SumValueSales
FROM cte_trimesters
WHERE [Brand] IN ('VANISH','LYSOL')
GROUP BY Trimester
ORDER BY AVG(TotalValueSales) DESC;
```

100 % 12 0

Results | Messages

	Trimester	Frecuencia	AvgUnitSales	AvgUnitWeeklySales	AvgValueSales	SumValueSales
1	Q3-2023	1078	2.144177	6.239510	90.360105	97408.194
2	Q2-2023	4767	1.908287	6.081772	82.832405	394862.075
3	Q1-2023	4881	1.998537	6.290706	77.432273	377946.925
4	Q4-2022	4953	1.773643	5.772343	71.966685	356450.993
5	Q3-2022	5307	1.756422	5.728780	68.891595	365607.697
6	Q2-2022	5625	1.566004	5.130912	61.516350	346029.473
7	Q1-2022	5215	1.492689	5.117998	57.630204	300541.514

Con estos resultados de esta última consulta, podemos observar que 2023 fue un mejor año de ventas comparado con 2022. Aunque el Q3 de 2023 destaca sobre los demás, por tener el mayor número de ventas monetarias y unitarias promedio, hay que tomar en cuenta que este trimestre solo cubre del 1 de julio al 17 de julio de 2023, por lo que solo está tomando en cuenta las promociones de verano, explicando así su alto rendimiento. Pasando a los trimestres más completos, el Q2 de 2023 es el mejor, superando incluso al Q4 de 2022 (donde hay ventas de invierno y navideñas). En general, se puede notar una tendencia de que las ventas de ambas marcas están creciendo desde inicios del 2022 hasta mediados de 2023, por lo que para los siguientes años es posible proyectar un aumento en las ventas.